

CSV → Kafka → Neo4j → Grounded AI Stack

Author: Tarun V • System: Full Docker Compose Stack • Date: September 2026

1. Executive Summary & Core Architecture

IntelliGuard is a production-grade, self-contained data streaming and graph query system. It enables non-technical users to drag-and-drop previously unseen CSV datasets of any size into a web application. The uploaded rows are instantly queued through **Apache Kafka**, ingested into **Neo4j Community Graph Database** via an idempotent streaming loader, and queried by a **Dynamic Schema-Aware Grounded Chatbot** that answers natural language questions strictly using Cypher queries without artificial hallucination.

Service	Technology	Port	Role & Responsibility
UI	Nginx + Vanilla JS	3050	Light-theme responsive web interface, Drag & Drop CSV, live polling progress bar, 20-row preview, grounded chat feed.
API	FastAPI (Python 3.11)	8001	Ingest endpoint, Kafka producer, schema introspection, Cypher translation, health checks.
Kafka	Apache Kafka 3.7.0	9092	High-throughput message broker in KRaft mode (no ZooKeeper required) for decoupled async streaming.
Loader	Python Kafka Consumer	Internal	Consumes row messages, performs idempotent MERGE into Neo4j, posts batch progress callbacks.
Neo4j	Neo4j 5.24 Community	7474 / 7687	Graph database storing :Dataset and :Row nodes with dynamic property sets and indexing.

2. Ingestion Pipeline & Graph Data Model

Deterministic Idempotency Key: Every upload computes a SHA-256 hash of filename + content, generating a stable 12-character dataset_id. Each row receives a composite MERGE key {dataset_id}:{row_index}. Re-uploading the identical CSV or replaying the Kafka topic produces zero duplicate graph nodes.

Node Label	Properties	Idempotency / Constraint Rule
:Dataset	id, filename, uploaded_at	MERGE (d:Dataset {id: \$dataset_id})
:Row	key, dataset_id, row_index, ...all_csv_columns	MERGE (r:Row {key: \$key}) SET r += \$columns
[:HAS_ROW]	Directed relationship (d)-[:HAS_ROW]->(r)	Ensures clean parent-child hierarchy for all records.

3. Dynamic Schema-Aware Chatbot Engine

The chatbot engine operates on a strict **groundedness-first** principle. Instead of generating unverified text from general memory, the chatbot inspects Neo4j's schema, maps user natural language intents to parameterised Cypher queries, and executes them against live data.

Feature	Engine Implementation	Benefit
---------	-----------------------	---------

Schema Introspection	Dynamically reads property keys from :Row nodes upon request.	Instantly supports any arbitrary CSV structure without manual configuration.
Synonym Resolution	Maps synonyms: <i>units/items</i> → quantity, <i>cost/dollars</i> → price, <i>client</i> → customer_id.	High conversational tolerance for varied user terminology.
Entity Value Lookup	Scans graph property indices for mentioned tokens (e.g. Widget A, C7, pending).	Automatic filtering without requiring explicit column names.
Transparent Proof	Every bot message returns the exact Cypher query and raw JSON result in an interactive detail drawer.	100% auditability; zero hallucination.
Honest Refusal	If an entity or column is absent from the CSV, returns grounded: false with available columns listed.	Prevents misleading responses for missing data.

4. End-to-End Test & Acceptance Verification Matrix

The system underwent thorough automated and manual acceptance tests with clean, large, and hostile datasets:

#	Question / Action	Generated Cypher	Actual Response	Status
1	Row count query	MATCH (r:Row) RETURN count(r) AS total	"There are 15 rows in the dataset."	PASS ✓
2	Filter by status	MATCH (r:Row) WHERE r.status = 'shipped' RETURN r	Found 5 row(s) for 'shipped' (status).	PASS ✓
3	Multi-word entity search	MATCH (r:Row) WHERE r.product = 'Widget A' RETURN sum(r.quantity)	There are 12.0 total units for 'Widget A'.	PASS ✓
4	Aggregate average	MATCH (r:Row) RETURN avg(toFloat(r.price)) AS avg	The average price is 37.32.	PASS ✓
5	Group by breakdown	MATCH (r:Row) RETURN r.category, count(r) ORDER BY count DESC	Electronics: 8, Hardware: 5, Software: 2.	PASS ✓
6	Missing entity refusal	None (Ungrounded)	"The entity 'units dollars' was not found. Available columns: ..."	PASS ✓
7	Idempotent Re-upload	MATCH (r:Row) RETURN count(r)	Row count remains strictly 15 (no duplicates).	PASS ✓
8	Hostile Empty CSV	POST /ingest (0 bytes)	HTTP 400 Bad Request with clear error banner.	PASS ✓
9	Healthcheck Endpoint	GET /health	{"status":"ok", "kafka_connected":true, "neo4j_connected":true}	PASS ✓

5. UI Design System & User Experience

The frontend is implemented with high-performance **Vanilla CSS and JavaScript** served via Nginx with automated API reverse proxying:

- **Modern Light Theme:** Cohesive palette featuring sky-blue accents (#2563eb), soft background glass (#f0f4ff), and crystal-clear slate typography (#0f172a).
- **Interactive Ingestion:** Drag-and-drop file zone with immediate client-side parsing displaying the first 20 rows in a sticky-header data table.
- **Live Streaming Progress:** Real-time progress bar polling GET /status at 1 Hz with percentage, row counters, and status badges (queued → loading → complete).
- **Audit-Ready Chat Experience:** Chat bubbles with groundedness badges (✓ Grounded in green, ✗ Not grounded in red) and collapsible Cypher + JSON result details.

6. Deployment & Reproduction Guide

The entire stack is configured for **1-command startup** via Docker Compose:

1. Clone & Navigate

```
git clone git@github.com:tarunv307/IntelliGuard.git
cd IntelliGuard/csv-graph-chat
```

2. Launch Entire Containerized Stack

```
docker compose up --build -d
```

3. Access Application

- Web UI: **http://localhost:3050**
- API & Swagger Docs: **http://localhost:8001/docs**
- Neo4j Browser: **http://localhost:7474** (User: *neo4j* / Password: *csvgraphdb*)

GitHub Repository: <https://github.com/tarunv307/IntelliGuard>